

Launder less

“The problem is that we attempt to solve the simplest questions cleverly, thereby rendering them unusually complex. One should seek the simple solution.”

— Anton Chekhov

Changes since [P3006R0](#) are highlighted with blue.

I. Motivation

```
alignas(T) std::byte storage[sizeof(T)];  
::new (&storage) T();  
// ...  
T *ptr_ = reinterpret_cast<T*>(&storage); // UB
```

The object that is nested within the storage array is not pointer-interconvertible with it [\[basic.compound\].p4](#). According to the Standard users have to call `std::launder` to avoid UB.

That behavior is surprising for the language users. Moreover, popular compilers do not require `std::launder` call in that place and produce the expected assembly without it.

This proposal attempts to remove UB in that place, standardize existing practice and simplify language usage.

II. More motivating examples

- `boost::optional` uses aligned storage and placement new to store a value. Storing a pointer from placement new may increase the size of an class.
- [utils::FastPimpl](#) from the userver framework uses byte array and placement new to store a value. Storing a pointer from placement new increases the size of an class.
- [HashTable](#) of the ClickHouse project. Storing a pointer from placement new increases the size of an class.
- [libcxx](#) in function.h.
- [V8](#) uses reinterpret casts and placement new. Storing the pointer increases object size.
- ...

`std::launder` is required to fix the above cases. However it is counterintuitive, decreases the code readability and requires in-depth knowledge of the Standard to realize that there is an issue from the point of the Standard (but in practice there is no problem!).

III. Impact on the optimizers

Due to many popular projects do cast storage to an object without calling `std::launder` and everything works fine - there's no optimization that is enabled by default and that relies on that particular UB in the Standard.

LLVM has a `-fstrict-vtable-pointers` optimization that is disabled by default. With that optimization turned on the `std::launder` for polymorphic objects emits a special barrier that takes over the argument, as a result it prevents some of the CSE optimizations (for example - prevent removal of loads). For non polymorphic types it does nothing.

However, note that placement new of dynamic object already emits a barrier according to [Devirtualization in LLVM and Clang](#). So using `std::launder` on a result of `reinterpret_cast` only adds an unnecessary optimization barrier.

IV. Wording

Adjust [\[expr.static.cast\] p14](#):

A prvalue of type “pointer to cv1 `void`” can be converted to a prvalue of type “pointer to cv2 T”, where T is an object type and cv2 is the same cv-qualification as, or greater cv-qualification than, cv1. If the original pointer value represents the address A of a byte in memory and A does not satisfy the alignment requirement of T, then the resulting pointer value is unspecified. Otherwise, if the original pointer value points to an object a, and there is an object b of type similar to T that is pointer-interconvertible with a, the result is a pointer to b. Otherwise, if the original pointer value points to an element of an array of `std::byte` or `unsigned char`, and there is an object b of type T for which the array provides storage, created at the address of the array element, the result is a pointer to b. Otherwise, the pointer value is unchanged by the conversion.

```
[Example 3:  
  T* p1 = new T;  
  const T* p2 = static_cast<const T*>(static_cast<void*>(p1));  
  bool b = p1 == p2;  // b will have the value true.  
- end example]
```

```
[Example 4:  
  alignas(T) std::byte storage[sizeof(T)];  
  auto* p1 = ::new (&storage) T();  
  auto* p2 = reinterpret_cast<T*>(&storage);  
  bool b = p1 == p2;  // b will have the value true.  
- end example]
```

IV. Acknowledgements

Many thanks to Артём Колпаков (Artyom Kolpakov) for asking the question on that topic at [STD discussion list](#) and for drawing my attention to the problem. Thanks to Brian Bi for answering the question at [STD discussion list](#).

Thanks to Andrey Erokhin, Timur Doumler, Roman Rusyaev, Mathias Stearn and all the mailing lists members for feedback and help!

Thanks to Andrey Erokhin and Jason Merrill for the wording help in R0!